

You Dream, We Display.

CH01

git hero lolms.page
gitglet conacity

Git과 GitLab 협업의 기준 좌표

Git 핵심 개념 · 역할별 협업 · 4공간 모델 · 개발환경 실습

👤 발표자명 | 🏢 발표부서 | 📅 2026-04-10



이 장에서 배울 내용



핵심 용어 정리

origin, HEAD, main, repository 등 필수
개념 정의



Git vs GitLab 역할 구분

VCS와 통합 협업 플랫폼의 차이 이해



역할별 협업 관점

Owner · Maintainer · Developer 권한 구분



4공간 모델 이해

워킹트리 → 스테이징 → 로컬 → 원격 저장소
흐름

CHAPTERS

CH01

CH02

CH03

CH04

CH05

CH05

CH06

CH07

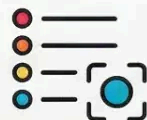
CH08

CH08



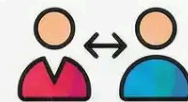
8챕터 전체 로드맵

- CH01~CH04: Git 핵심 기초 구간
(환경 설정, 상태 변화, 이력 조회,
브랜치 전략 순서 진행)
- CH05~CH06: GitLab 팀 협업 구간
(실전 MR 프로세스, 승인 규칙,
충돌 해결 실습 중심)
- CH07~CH08: CI/CD 및 확장 구간
(파이프라인 설계, Quality Gate,
MLOps 관점 완성)



세부 학습 포인트 (챕터별)

- Git 핵심: 상태 진단, 변경 이력 및
브랜치 최적화 운영
- GitLab 협업: MR 승인 프로세스,
팀 통합 품질 및 갈등 해결
- CI/CD 및 확장: 자동 파이프라인
구축 및 MLOps 관점 통합



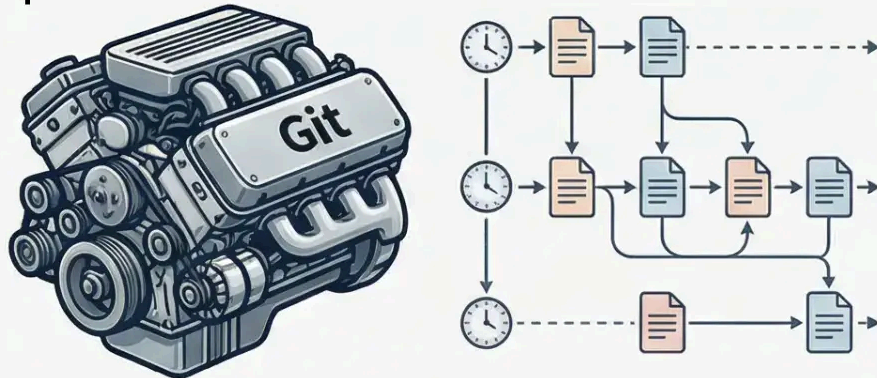
역할별 렌즈 비교

- Owner 렌즈: 프로세스 및 정책
안정성 확보, Protected Branch
및 승인 규칙 설계 책임
- Maintainer 렌즈: 통합 품질 확보,
리뷰 및 병합 관리 집중
- Developer 렌즈: 로컬 개발 상태
진단, 원격 동기화 클리너 최적화에
집중

Git vs GitLab 버전 관리와 협업 운영의 차이 / Git vs GitLab Differences

Git (로컬 버전 관리 엔진)

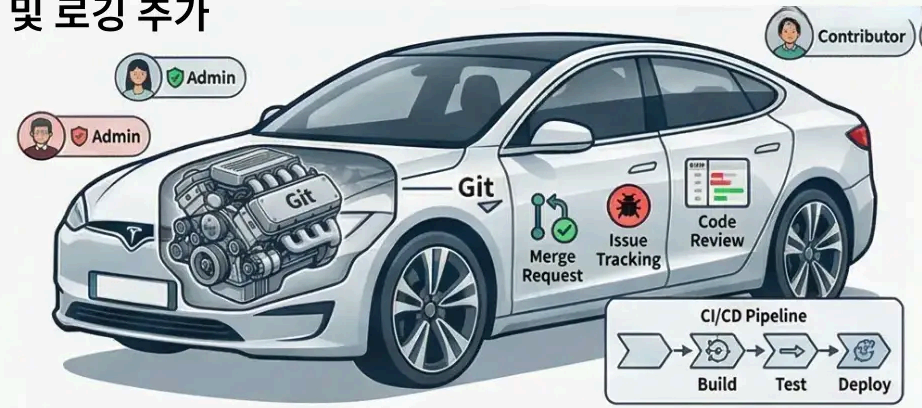
- 로컬 버전 관리 핵심 도구: 파일 변경 이력을 로컬에서 추적 및 기록
- 오프라인 동작 가능: 네트워크 없이 로컬 저장소에서 commit-branch 작업
- 개인 최적화: 개인 작업 및 변경 사항의 빠르고 효율적인 관리



Git: 엔진 비유 (핵심 도구)

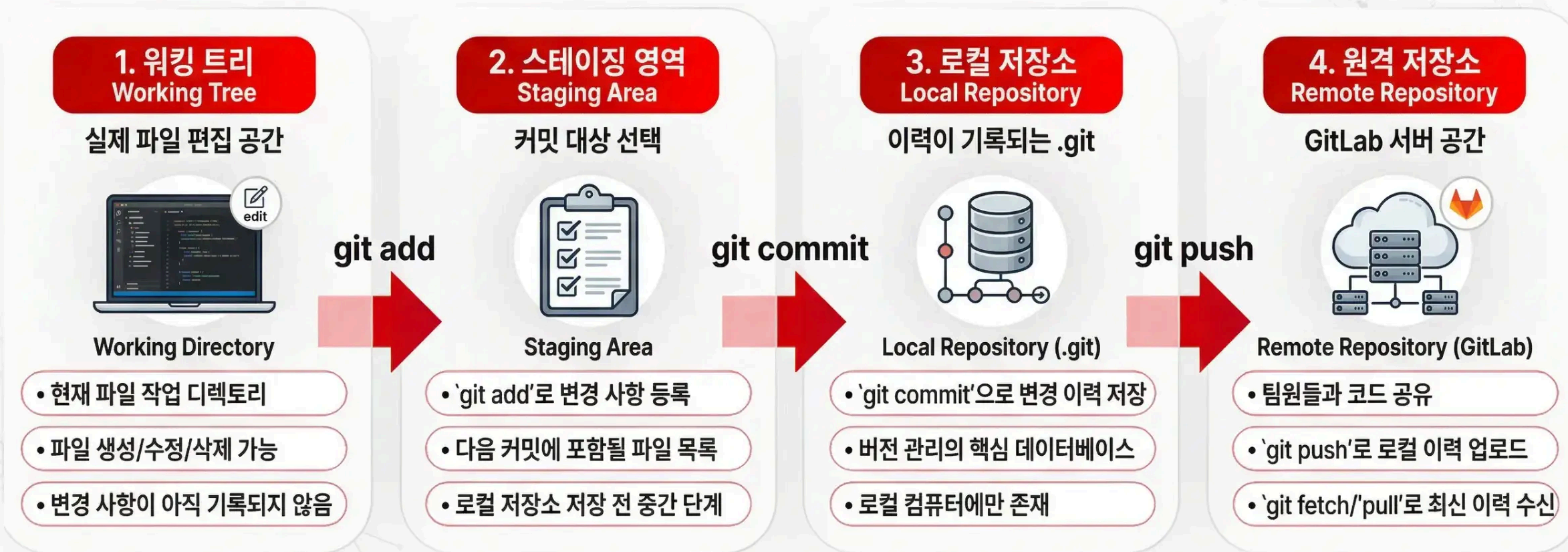
GitLab (협업 운영 플랫폼)

- 협업 운영 플랫폼 (웹 서비스): 권한 관리, 코드 리뷰 (MR), CI/CD 기능 제공
- 팀 통합 레이어: MR, Approval, 이슈 트래킹으로 팀 작업을 안전하게 통합
- 제어 및 감사 기능: Git을 팀 단위로 안전하게 쓰도록 제어 및 로깅 추가

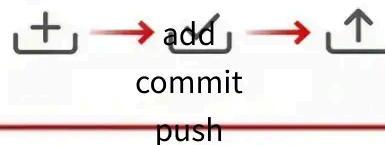


GitLab: 자동차 비유 (운영 플랫폼)

4가지 핵심 Git 공간과 데이터 이동 프로세스



데이터 이동 순서



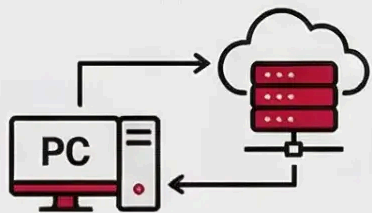
흔한 오해

commit ≠ GitLab 반영

- 로컬(commit까지)과 원격(push 이후)의 명확한 구분 필요.



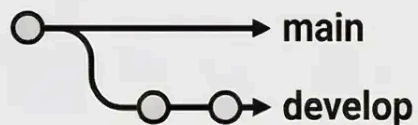
저장소와 origin



Repository: 프로젝트
이력 저장 단위 (.git
폴더 포함)

origin: 클론 시 지정되는
원격 저장소 이름

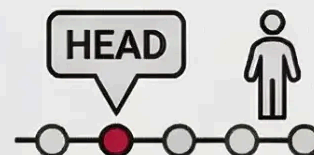
주요 브랜치 구성



기본 브랜치: main
(과거 master) 또는
develop 사용

명칭은 프로젝트별 상이,
git branch -vv로 확인

HEAD 포인터



현재 작업 위치를
가리키는 포인터

내가 보고 있는 브랜치
또는 특정 커밋을 지시

Detached HEAD

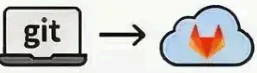


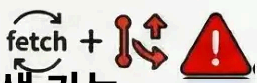
특정 커밋 직접
체크아웃 시 발생

커밋 유일 위험 주의!
(새 브랜치 생성 권장)

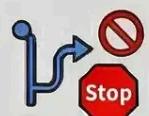
Git 기초 상식 교정 / Git Foundation Facts Correction

Git 오해와 진실 TOP 6

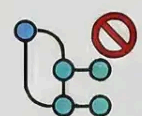
 오해: 커밋하면 GitLab에 반영된다 
→ 진실: 커밋은 로컬 기록, push 후 원격 반영

 오해: pull은 항상 안전한 명령이다 
→ 진실: pull = fetch + merge, 충돌 발생 가능

 오해: add는 파일을 저장하는 것이다 
→ 진실: add는 커밋 대상 목록에 추가하는 것

 오해: main 브랜치에 바로 push가 편하다
→ 진실: MR 리뷰 없는 직접 push는 협업에서 위험

오해: origin은 GitLab 자체를 의미한다
→ 진실: origin은 원격 저장소 URL의 별칭(별명)

 오해: HEAD는 항상 최신 커밋이다
→ 진실: HEAD는 현재 작업 위치 포인터, 과거 커밋 가능

핵심 요약: Git은 로컬 중심의 분산 버전 관리 시스템입니다.
pull 전 충돌을 독해 점검하고, MR을 통해 협업 품질을 높이세요.



로컬 상태 진단을 위한 기본 명령어 세트

`git status` — 현재 작업 트리 상태 확인

수정·추가·삭제된 파일 목록과 스테이징 여부를 한눈에 파악

`git branch -vv` — 브랜치 추적 관계 확인

로컬 브랜치가 origin/main을 올바르게 추적하는지 검증

`git remote -v` — 원격 저장소 URL 확인

origin 주소와 HTTPS-SSH 인증 방식을 확인하는 기준 명령

`git log --oneline` — 커밋 이력 요약 조회

최근 커밋 메시지와 메시지를 한 줄씩 출력해 이력 흐름 파악

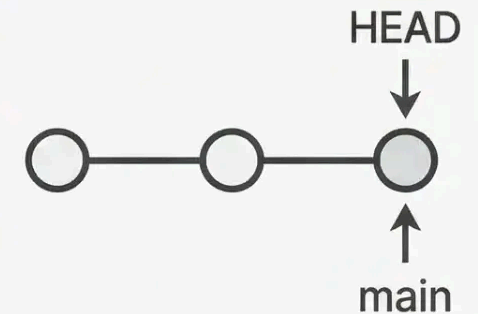
`git config --list` — 전체 Git 설정 일괄 확인

user.name-user.email 등 로컬·글로벌 설정값을 한 번에 점검

✓ HEAD

관찰 포인트: HEAD가
가리키는 위치

현재 체크아웃된 브랜치 또는
커밋을 파악해 작업 기준점 설정



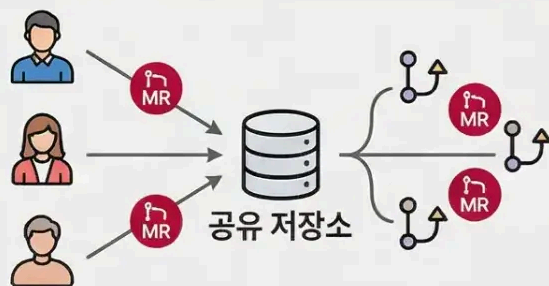
Shared Repository와 Fork 모델 비교

사내 협업 vs 오픈소스 기여 방식 비교

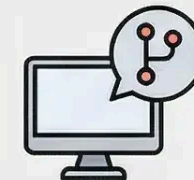


사내 협업 표준: 공유 저장소 모델

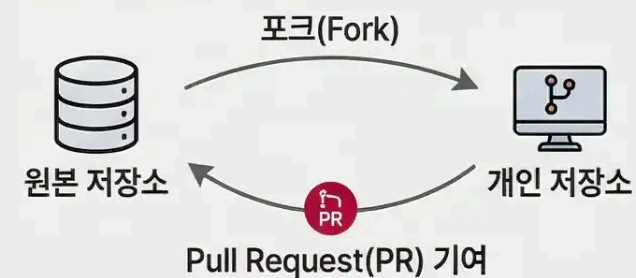
- ✓ 중앙 집중형 저장소 사용
- ✓ Feature 브랜치 기반 작업
- ✓ Merge Request(MR)를 통한 코드 통합
- ✓ 강력한 역할 및 권한 관리
- ✓ 본 교육 과정의 기준 좌표



오픈소스 기여 표준: 포크 모델



- ✓ 원본 저장소의 복제본(fork) 생성
- ✓ 개인 계정 저장소에서 작업
- ✓ Pull Request(PR)로 변경 사항 기여
- ✓ 외부 기여자와의 협업에 적합
- ✓ 프로젝트 성격에 따라 선택



GitLab 권한 정책과 역할별 책임 Matrix

CH01 젠더 배지 | 1단계
핵심 파트너

You Dream,
We Display.

소유자 (Owner)



• 최상위 정책 설계자

- Protected Branch 설정, 멤버 권한 부여, 외부 도구 통합 등 저장소 전체 운영 정책을 결정하고 관리합니다.

관리자 (Maintainer)



• 품질 및 방향 책임자

- MR 최종 승인 및 병합(Merge) 권한을 독점적으로 보유하여, 코드 품질과 브랜치 안정성을 실질적으로 책임집니다.

개발자 (Developer)



• 기능 구현의 실천자

- 기능 개발 브랜치 생성 및 Push, MR 요청이 가능합니다. 단, main 브랜치 직접 Push는 제한되어 브랜치 보호를 실현합니다.

권한 운영 핵심 가이드



방법 권한 ≠ 기술 속력도:



Protected Branch와 연동: main 브랜치는 Maintainer 이상으로 제한하여 신뢰성 확보.

초기 클론 상태 및 설정 확인

TITLE

1. Git 환경 및 정보

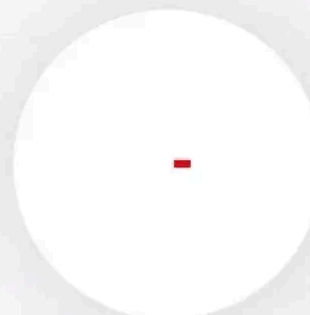
git --version 및
설치 상태 점검
git config --global
user.name/email
설정 점검

2. 저장소 클론 및 이동
git clone <repo-url>
실행
cd <repo-name>
디렉토리 이동

6. 실패 시그널 및 조치

명령어 오류 시 점검 순서: 1) Git 미설치, 2) 사용자 정보
누락, 3) 인증 실패

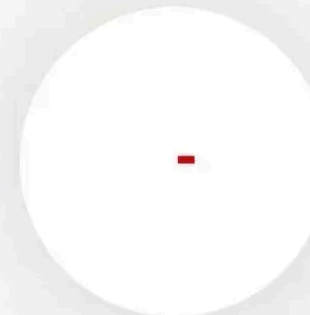
3. 브랜치 상태



1) git status로
워킹 트리 점검

2) 및 현재 브랜치 확인

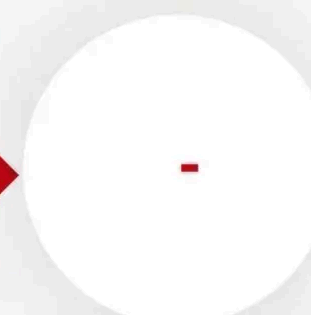
4. 추적 관계



1) git branch -vv
로컬 추적 확인

2) origin/main
정상 추적 점검

5. 원격 URL & 인증



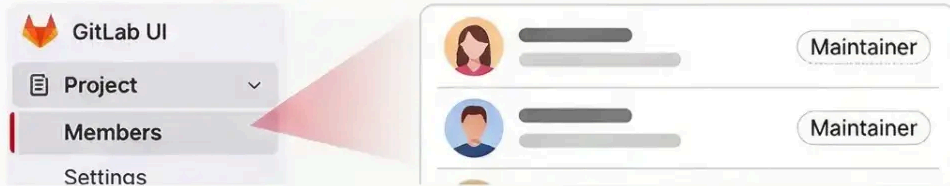
1) git remote -v
URL 주소 확인

2) HTTPS/SSH
인증 방식 점검

GitLab UI를 활용한 역할 및 보안 정책 확인 방법

권한 항목	Owner (소유자)	Maintainer (관리자)	Developer (개발자)	Guest (손님)
프로젝트 관리 (Setting, Integrations)	✓ 권한 관리	✓ 관리자	✓ 개발자	✓ 손님
코드 커밋 & 머지 (Protected Branches)	✓ 코드 커밋	✓ 코드 커밋	✓ 코드 커밋	✓ 손님
보호 브랜치 설정	✓ 보호 설정	✓ 보호 설정	-	-
MR 승인 규칙 설정	✓ 승인 설정	✓ 승인 설정	✓ 승인 설정	-
멤버 추가/삭제	✓ 멤버 추가	✓ 멤버 추가	-	-
CI/CD 설정	✓ 자동화 파이프라인 구성 및 관리	✓ 자동화 파이프라인 구성 및 관리	✓ 자동화 파이프라인 구성 및 관리	-

1. 프로젝트 멤버 역할 조회



좌측 메뉴 → [Project] → [Members] 에서 내 역할 조회

2. Protected Branches 및 승인 규칙 확인



3. 정책 vs 명령어 오류 구분



핵심 요약 및 차기 학습 안내

1 CH01 주요 학습 요약

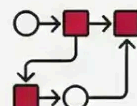
Git vs GitLab 역할 차이
(로컬 vs 협업 플랫폼)

4단계 작업 공간 흐름 파악
(워킹트리→스테이징→로컬→원격)

Git 사용자 정보 설정 확인
(`'user.name'`, `'user.email'`)

clone 후 상태 진단 명령어
(`'status'`, `'branch -vv'`, `'remote -v'`)

GitLab 역할 및 보호 정책
(권한 관리 및 main 보호)



2 차기 CH02 실습 예고



파일 수정



`'git add'`



`'git commit'`



`'git push'`

첫 번째 변경 사이클 실습 완